```
enum WizardSpells: string implements CastSpell {
    case MagicMissile = 'Thunk! Thunk! Thunk!';
    case Thunderwave = 'KABOOM!';

    public function cast(): string
    {
        return $this->value . PHP_EOL;
    }
}

function doMagic(CastSpell $spell)
{
    print "You cast a spell, it goes " . $spell->cast();
}

doMagic(WizardSpells::MagicMissile);
doMagic(WizardSpells::Thunderwave);

Output:
You cast a spell, it goes Thunk! Thunk! Thunk!
You cast a spell, it goes KABOOM!
```

Here we have a `CastSpell` interface implemented by the `WizardSpells` enum. `WizardSpells`, just like a class, must provide the interface's `cast()` implementation. The function `doMagic()` accepts a single argument `$spell`, which must be an object that implements the `CastSpell` interface, and `WizardSpells` passes the type check.

What is important to note here is the call to the `cast()` method: it is called on the enum cases, not the enum itself. Remember, it is the cases that are the objects. Conceptually, you can think of `WizardSpells::MagicMissile` as being functionally equivalent to `abstract class WizardSpells` and `class MagicMissile extends WizardSpells`. Therefore, when we call `WizardSpells::MagicMissle->cast()` the value of `$this` in the `cast()` method is set to `WizardSpells::MagicMissile`.

Pure and backed enums can implement interfaces (the `implements` keyword goes after the type declaration for backed enums) and define methods (either their own or those specified by an interface). An enum may not have properties, however, nor can they be extended. Although protected methods are legal in an enum declaration, they can never be inherited, so stick to `public` and `private` visibility.